Ouvidoria Inteligente

Relatório de implementação e estado da entrega

Estado parcial. A aplicação, os dados e os três notebooks estão implementados. As análises experimentais ainda precisam ser executadas até o fim. Este documento registra o que está pronto e as pendências, sem apresentar métricas que não foram calculadas.

Objetivo e arquitetura

O projeto compara BoW, TF-IDF e embeddings multilíngues, busca manifestações por cosseno, sugere duplicatas e experimenta chunking. app_ouvidoria.py reúne a interface Streamlit; ouvidoria_core.py fornece validação da base, codificação, divisão em chunks, projeções e avaliação de pares. Os três notebooks compartilham esse núcleo.

Base demonstrativa autorizada

manifestacoes.json contém 40 registros sintéticos, com exatamente os campos id, data, categoria_oficial e texto. Os textos têm de 50 a 800 caracteres. M010, M016, M024, M033 e M040 ultrapassam 500 caracteres. As categorias são infraestrutura, saúde, segurança, educação e meio ambiente. A base não representa cidadãos reais e não corresponde à base original do professor.

O gabarito duplicatas_reais.json contém M003-M017, M008-M022 e M005-M026. Os três pares envolvem seis registros, equivalentes a 15% da base. A convenção de contagem e a origem dos dados estão em PROVENIENCIA.json. A duplicidade é definida por evento, local e período compatíveis, não apenas por categoria.

Funcionalidades implementadas

A aplicação possui abas de busca, base completa, espaço vetorial e chunking. A sidebar permite selecionar MiniLM multilíngue ou DistilUSE multilíngue e alterar top-k. Os modelos e embeddings usam cache. As cores dos resultados seguem os limites 0,7 e 0,5 do enunciado. PCA e t-SNE são selecionáveis; o laboratório de chunking mostra trechos e vetores.

Textos acima da capacidade do modelo são divididos em janelas de tokens, codificados e agregados por média normalizada. O experimento de chunking usa RecursiveCharacterTextSplitter com (240, 0), (240, 60) e (400, 80), medindo sobreposição efetiva e similaridade dos vizinhos.

Validação realizada

Cinco testes de lógica da Ouvidoria passaram na execução anterior: seleção de pares sem diagonal, limiar estrito, precisão e recall em casos conhecidos, cobertura e offsets dos chunks, projeção com um ponto e validação do JSON, agrupados em cinco testes. A estrutura da base também foi verificada quanto a IDs, campos e limites de tamanho.

O primeiro carregamento da interface e os embeddings do MiniLM foram exercitados. A bateria completa da interface não terminou; o roteiro de testes foi corrigido para selecionar widgets pelo rótulo, mas ainda precisa de uma execução final. Nenhum resultado de duplicatas ou chunking é declarado validado neste relatório.

Pendências para concluir o enunciado

1. Concluir o download dos dois modelos e executar análise_comparativa.ipynb, gerando a tabela dos três pares obrigatórios.
2. Executar deteccao_duplicatas.ipynb para obter heatmap, pares detectados, falsos positivos, falsos negativos e análise de limiar.
3. Executar chunking_manifestacoes.ipynb, inspecionar os limites reais dos trechos e justificar a configuração com exemplos e projeções.
4. Concluir os testes da interface e substituir este documento por um relatório final de até cinco páginas.

Obstáculos e reprodução

A execução foi interrompida por falha de conexão no download do DistilUSE e por arquivos do ambiente tornados disponíveis apenas online pelo iCloud. Para continuar, recomenda-se criar um novo ambiente Python 3.12 fora da pasta sincronizada, instalar requirements.txt e executar os notebooks na pasta do projeto. HF_HUB_DISABLE_XET=1 permite tentar um método alternativo de transferência dos pesos, se necessário.

A aplicação inicia com **streamlit run app_ouvidoria.py**. O primeiro uso exige baixar os modelos públicos; as manifestações são processadas localmente. Os notebooks entregues contêm o código, mas estão sem saídas executadas. CONTINUIDADE.md contém instruções para outra IA, critérios de aceitação e o estado exato de cada projeto.

Autoria e referências

Trabalho individual de Fernando Amorim Pontes Neto. A base sintética foi autorizada pelo solicitante. Referências: Desafio_Ouvidoria_Inteligente.docx; materiais de apoio enviados; documentação de Sentence Transformers, LangChain Text Splitters e scikit-learn.